

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Carrera de Ingeniería de Software

SEGUNDO AVANCE DEL PROYECTO FINAL

Pruebas unitarias, de integración, funcionales, de rendimiento y de seguridad básica para validar funcionalidades del sistema de gestión veterinaria Spring Petclinic

Curso:
Pruebas de Software

Docente:
Ing. Mgtr. Jonathan Merino Farias

Integrantes:
Marcelo Fabrizio Araujo Rosas
Alvaro Christian Pilco Gallardo
Helam Rojas Ramón

Lima – Perú
2026

Índice

1. Introducción
2. Planteamiento del problema
3. Justificación
4. Objetivos
5. Sistema, alcance y limitaciones
6. Marco teórico y metodología
7. Requisitos y trazabilidad
8. Plan de pruebas
9. Diseño de casos de prueba
10. Pruebas unitarias e integración
11. Pruebas funcionales
12. Pruebas de rendimiento básico
13. Pruebas de seguridad básica
14. Pruebas móviles, de usabilidad y accesibilidad (hito posterior)
15. Pruebas de regresión (hito posterior)
16. Registro de incidencias y correcciones
17. Métricas y análisis de resultados
18. Conclusiones (hito posterior)
19. Referencias
20. Anexos

1. Introducción

El proyecto evalúa Spring Petclinic, una aplicación web desarrollada en Java con Spring Boot para gestionar propietarios, mascotas y visitas de una clínica veterinaria. Se mantienen las seis funcionalidades del primer avance: registrar, buscar y actualizar propietarios; registrar y actualizar mascotas; y registrar visitas.

Este segundo avance actualiza las secciones 1 a 10 y documenta la ejecución de los casos CP01 a CP06, el análisis y la corrección de las dos fallas iniciales, las pruebas funcionales, una prueba básica de rendimiento y una revisión básica de seguridad sobre una instancia local. También registra las incidencias encontradas y las métricas obtenidas.

La ejecución consolidada se realizó el 29 de octubre de 2026 sobre el commit 500158f del repositorio oficial, con MySQL 8.0.46 como base de datos principal y H2 como base de comparación. Los resultados se refieren a los escenarios ejecutados y no acreditan la ausencia de defectos en el resto del sistema.

2. Planteamiento del problema

2.1. Descripción del problema

La gestión de propietarios, mascotas y visitas requiere datos válidos y asociaciones consistentes. Un registro incompleto o una relación incorrecta puede afectar la consulta y actualización de la información. Se evalúa si las funcionalidades seleccionadas cumplen los requisitos definidos, sin asumir previamente la existencia de defectos.

Los antecedentes citados abordan la evaluación de sistemas de información en salud (Noël et al., 2022), la calidad de los artefactos de prueba (Tran et al., 2021), las pruebas de integración (Shin et al., 2013) y las prácticas de regresión (Minhas et al., 2020). Sus contextos difieren del presente proyecto; se emplean para organizar la evaluación, no como evidencia de fallos en Spring Petclinic.

2.2. Formulación del problema

¿En qué medida las funcionalidades seleccionadas de Spring Petclinic cumplen los requisitos de validación, almacenamiento y asociación de datos al evaluarse mediante pruebas unitarias, de integración, funcionales, de rendimiento básico y de seguridad básica?

3. Justificación

3.1. Justificación técnica

Las pruebas permiten contrastar el comportamiento de las funcionalidades con criterios explícitos. La evaluación distingue las validaciones de componentes individuales, las operaciones que requieren la base de datos y los flujos completos accesibles por HTTP. Esta separación facilita identificar el alcance de cada caso y la causa de cada diferencia.

3.2. Justificación académica

El proyecto aplica el diseño, la ejecución y la documentación de pruebas sobre una aplicación existente. Relaciona requisitos con casos, evidencias e incidencias, y distingue entre un defecto del producto, un problema del instrumento de prueba y un problema del ambiente.

3.3. Justificación práctica

Los scripts, reportes y la matriz editable permiten repetir las verificaciones y comprobar las correcciones. El trabajo se delimita a las funcionalidades seleccionadas para mantener una evaluación viable con los recursos y el periodo académico disponibles.

4. Objetivos

4.1. Objetivo general

Evaluar el cumplimiento de los requisitos de las funcionalidades seleccionadas de Spring Petclinic mediante pruebas unitarias, de integración, funcionales, de rendimiento básico y de seguridad básica.

4.2. Objetivos específicos

- Diseñar y ampliar casos de prueba vinculados con los requisitos de propietarios, mascotas y visitas.
- Ejecutar pruebas unitarias y de integración, analizar sus fallas y comprobar las correcciones.
- Ejecutar pruebas funcionales con datos válidos, inválidos y valores límite sobre los seis flujos.
- Medir tiempos de respuesta y errores bajo una carga local acotada.
- Verificar controles de seguridad pertinentes: autenticación existente, validación de entradas, asociación propietario-mascota y exposición de información.
- Registrar incidencias y métricas con evidencias verificables.

5. Sistema, alcance y limitaciones

5.1. Descripción del sistema

Spring Petclinic se usa como sistema existente bajo prueba, sin modificar su código de producción. Las funcionalidades seleccionadas se agrupan en dos módulos.

Módulo funcional	Funcionalidades seleccionadas
Gestión de propietarios	Registro, búsqueda por apellido y actualización de datos.
Gestión de mascotas y visitas	Registro y actualización de mascotas asociadas a un propietario; registro de visitas para una mascota.

5.2. Reglas de negocio y validaciones

Antes de ejecutar se contrastó cada regla con el código del commit evaluado. La columna de verificación indica dónde se implementa; el resultado de las pruebas se presenta en las secciones 10 a 13.

Código	Regla	Implementación verificada en el código
RN01	Dirección, ciudad y teléfono del propietario obligatorios; teléfono de exactamente diez dígitos.	Owner: @NotBlank y @Pattern(\d{10})
RN02	Toda mascota debe asociarse a un propietario existente.	Relación Owner–Pet (@OneToMany, owner_id)
RN03	La mascota requiere nombre, tipo y fecha de nacimiento.	PetValidator
RN04	Sin nombres de mascota duplicados para un mismo propietario.	PetController y restricción única (owner_id, name)
RN05	La fecha de nacimiento no puede ser posterior a la fecha actual.	PetController (alta y edición)
RN06	La visita se asocia a una mascota del propietario indicado y tiene descripción no vacía.	VisitController y Visit: @NotBlank
RN07	La fecha de una nueva visita debe ser posterior al día actual.	VisitController: rechaza hoy y fechas pasadas

5.3. Alcance del segundo avance

El avance comprende la ejecución de CP01 a CP06 y su corrección, 31 casos funcionales (F01–F31), 13 verificaciones de seguridad básica (S01–S13), una prueba exploratoria de concurrencia (E01), dos escenarios de carga local y el registro de incidencias y métricas. Las secciones 14, 15 y 18 corresponden a hitos posteriores.

5.4. Limitaciones del proyecto

- **Ambiente.** Las ejecuciones se realizaron en un contenedor Linux local (2 vCPU, 7,8 GiB). La ejecución de transferencia usó Windows con Temurin 21.0.13+9; la consolidada usó OpenJDK 21.0.12.1. Ambas cumplen el requisito java.version=17 del proyecto.
- **Base de datos.** El plan preveía MySQL. Se instaló MySQL 8.0.46 y se ejecutaron sobre él las pruebas de integración, funcionales, de seguridad y de rendimiento. H2 se usó como comparación; sus resultados se identifican por separado.
- **Seguridad.** La variante evaluada no implementa autenticación ni autorización. Las pruebas se limitan a la instancia local y no constituyen una auditoría de seguridad.
- **Rendimiento.** El generador de carga y la aplicación compartieron el mismo equipo. Los tiempos no representan la capacidad de un entorno de producción.

- **Pruebas manuales.** Los flujos de interfaz se recorrieron con un navegador automatizado. No se registraron pruebas manuales realizadas por integrantes del equipo.
- **Responsables.** La asignación nominal de responsables por caso no consta en las fuentes recibidas y queda pendiente de registro por el equipo.

6. Marco teórico y metodología

6.1. Conceptos fundamentales

Pruebas de software. Actividades que contrastan el comportamiento de un sistema con criterios definidos. Sus resultados aportan evidencia sobre los escenarios examinados; no demuestran la ausencia de defectos.

Pruebas unitarias. Comprueban una unidad de comportamiento de forma aislada. Aquí se aplican a las validaciones del propietario y de la mascota.

Pruebas de integración. Comprueban la interacción entre componentes, en este caso repositorios JPA, base de datos y controladores (Shin et al., 2013).

Pruebas funcionales. Verifican un flujo completo desde la perspectiva del usuario, con datos válidos, inválidos y límites. Se ejecutaron por HTTP real contra la aplicación en marcha.

Pruebas de rendimiento. Miden tiempos de respuesta, rendimiento y errores bajo una carga definida.

Pruebas de seguridad básica. Verifican controles pertinentes al sistema, como validación de entradas, control de acceso y exposición de información, con referencia a categorías del OWASP Top 10 (OWASP Foundation, 2021).

Caso de prueba y trazabilidad. El caso define precondiciones, datos, pasos y resultado esperado; tras ejecutarlo se registran obtenido, estado y evidencia (Tran et al., 2021). La trazabilidad vincula requisitos, casos, resultados e incidencias.

Incidencia. Diferencia entre esperado y obtenido. Se analiza considerando requisitos, datos y ambiente antes de clasificarla como defecto del producto, problema del instrumento de prueba o problema del ambiente.

6.2. Metodología de trabajo

Se mantiene la adaptación de Scrum con seis sprints de dos semanas entre las semanas 6 y 17. Una tarea de ejecución se considera terminada cuando cuenta con resultado y evidencia; el diseño de un caso, por sí solo, no se registra como prueba ejecutada.

6.3. Planificación y estado de actividades

El estado corresponde al 29 de octubre de 2026, fecha de la ejecución documentada.

Sprint	Semanas	Actividades previstas	Estado
1	6–7	Completar unitarias e integración y sus evidencias. Ampliar casos e iniciar funcionales.	Realizado (ver secciones 9 a 11)
2	8–9	Ejecutar funcionales, registrar incidencias y comprobar correcciones.	Realizado
3	10–11	Pruebas básicas de rendimiento y seguridad.	Realizado
4	12–13	Consolidar el segundo avance (semana 12) e iniciar pruebas móviles.	APF2 consolidado; pruebas móviles pendientes
5	14–15	Usabilidad, accesibilidad y regresión.	Pendiente
6	16–17	Tercer avance y consolidación del informe final.	Pendiente

6.4. Organización del trabajo

Las modificaciones de casos se registran con su justificación. Las correcciones de las pruebas se documentan como incidencias del instrumento (INC-T01, INC-T02). Toda regla dependiente de la versión se vincula al commit 500158f732419217507c7656904b8e6aa1bcc0d6.

7. Requisitos y trazabilidad

7.1. Requisitos funcionales

Código	Requisito
RF01	Registrar propietarios y validar los campos obligatorios y el formato del teléfono.
RF02	Buscar propietarios por apellido y consultar su información.
RF03	Actualizar los datos de un propietario existente, conservando las validaciones del registro.
RF04	Registrar mascotas asociadas a un propietario y validar datos obligatorios, nombres duplicados y fecha de nacimiento.
RF05	Actualizar el nombre, tipo y fecha de nacimiento de una mascota, respetando las validaciones correspondientes.
RF06	Registrar visitas para una mascota del propietario indicado, con descripción no vacía y fecha posterior al día actual (RN07).

7.2. Matriz de trazabilidad actualizada

La cobertura se califica respecto de las reglas definidas. «Completa» indica que todas las reglas asociadas tienen al menos un caso ejecutado; no implica ausencia de defectos.

Req.	Casos	Cobertura	Pendiente o con hallazgo
RF01	CP01; F01–F06; S03, S05, S10	Parcial	Teléfono vacío y nombre vacío en el registro (el nombre vacío se probó en la actualización, F14)
RF02	CP02; F07–F11; S04, S09	Completa para el alcance definido	Navegación de páginas posteriores a la 2
RF03	CP03; F12–F14; S06	Parcial	Dirección o ciudad vacías durante la actualización
RF04	CP04; F15–F21; E01	Completa para RN03–RN05 en flujo secuencial	RN02 falla bajo envíos simultáneos (DEF-P02, DEF-P03)
RF05	CP05; F22–F25; S08	Completa para las reglas definidas	Manejo del error al editar una mascota ajena (DEF-P01)
RF06	CP06; F26–F31; S07	Completa para RN06 y RN07	Código HTTP 500 para mascotas no asociadas (OBS-02)

Estado de la última versión: 45 casos aprobados, 5 fallidos, 1 observación y 0 bloqueados, sobre 51 casos. La matriz detallada se encuentra en matriz/APF2_Matriz_Casos_Incidencias_Metricas.xlsx.

8. Plan de pruebas

8.1. Objetivo, alcance y estrategia

Verificar los requisitos seleccionados en cinco niveles: unitario (CP01, CP05), integración con repositorios y MockMvc (CP02–CP04, CP06), funcional por HTTP real (F01–F31), rendimiento básico y seguridad básica (S01–S13). Se usan datos válidos, inválidos y límites confirmados en el código.

8.2. Ambiente, herramientas y datos

Elemento	Configuración usada
Sistema	Spring Petclinic, commit 500158f732419217507c7656904b8e6aa1bcc0d6; Spring Boot 4.1.0; artefacto spring-petclinic-4.0.0-SNAPSHOT.jar (SHA-256 en evidencias/AMBIENTE.txt).
Entorno	Ubuntu 24.04.5 LTS, 2 vCPU, 7,8 GiB; OpenJDK 21.0.12.1; Maven Wrapper 3.9.16. Transferencia previa: Windows, Temurin 21.0.13+9.
Base de datos	MySQL 8.0.46 local con db/mysql/schema.sql y data.sql del proyecto (usuario petclinic). H2 2.4.240 como comparación.
Herramientas	JUnit 5, Spring Boot Test, MockMvc; Python 3.13 con requests (funcionales y seguridad); Playwright 1.63 con Chromium (navegador automatizado); Locust 2.47.0 (rendimiento).
Ejecución	./mvnw test con filtros por clase; scripts en scripts/; comandos en el Anexo B.
Control de versiones	Git. Repositorio del equipo: https://github.com/Mooneer17/spring-petclinic-apf2 (rama main), con el código, los scripts, las evidencias y los documentos de este avance.
Datos	Ficticios. Cada caso funcional crea sus propios registros con un sufijo único; las pruebas JPA revierten su transacción.

8.3. Criterios de inicio y finalización

Un caso es aprobado si el obtenido coincide con el esperado, fallido si difiere y bloqueado si una condición impide ejecutarlo. Se añade «Observación» para verificaciones del estado real sin criterio de aprobación, como la existencia de autenticación. La ejecución finalizó con todos los casos ejecutados y sin casos bloqueados.

8.4. Responsables y evidencias

Cada evidencia identifica el caso, la ejecución y la base de datos. La asignación nominal de responsables de ejecución y revisión queda pendiente de registro por el equipo en la matriz.

9. Diseño de casos de prueba

Se mantienen CP01 a CP06 con sus datos y resultados esperados del primer avance; se añaden el resultado obtenido, el estado y la evidencia de la última ejecución en MySQL. Los casos ampliados se describen en las secciones 11 y 13, y con todos sus campos en la hoja «Casos» de la matriz.

CP01 / RF01 · Unitaria

Campo	Descripción
Precondiciones y datos	Ana Torres, Av. Central 123, Lima; 987654321 y 9876543210
Procedimiento	Validar con Bean Validation el teléfono de 9 dígitos; repetir con 10 dígitos
Resultado esperado	9 dígitos: error en telephone; 10 dígitos: sin errores
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP02 / RF02 · Integración (repositorio JPA)

Campo	Descripción
Precondiciones y datos	Propietario TorresAPF guardado en la base de prueba
Procedimiento	Guardar, limpiar el contexto de persistencia y buscar por apellido
Resultado esperado	Un resultado con el mismo id, nombre y teléfono
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP03 / RF03 · Integración (repositorio JPA)

Campo	Descripción
Precondiciones y datos	Propietario existente; nuevo teléfono 9123456780
Procedimiento	Actualizar, guardar, limpiar el contexto y recargar por id
Resultado esperado	Mismo id y teléfono 9123456780
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP04 / RF04 · Integración (repositorio JPA)

Campo	Descripción
Precondiciones y datos	Mascota LunaAPF, 2022-05-10, primer tipo disponible
Procedimiento	Asociar, guardar, limpiar el contexto y recuperar por id de propietario y de mascota
Resultado esperado	Mascota no nula con nombre, fecha y tipo registrados
Resultado obtenido	Mascota recuperada con id asignado, nombre LunaAPF, fecha 2022-05-10 y tipo no nulo

Campo	Descripción
Estado	Aprobado (R3, MySQL). Primera ejecución: Fallido
Evidencia	evidencias/03_correccion_MySQL/surefire-reports; incidencia INC-T01

CP05 / RF05 · Unitaria

Campo	Descripción
Precondiciones y datos	Mascota con tipo cat y fecha 2022-05-10; nombre '' y 'Luna'
Procedimiento	Ejecutar PetValidator.validate() con nombre vacío y con Luna
Resultado esperado	Vacío: error en name; Luna: sin errores (no comprueba persistencia)
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP06 / RF06 · Integración (MockMvc + JPA)

Campo	Descripción
Precondiciones y datos	Mascota persistida; fecha de mañana; 'Control general APF'
Procedimiento	POST /owners/{id}/pets/{petId}/visits/new con MockMvc; recargar la mascota
Resultado esperado	Redirección 3xx; una visita con la fecha y descripción enviadas
Resultado obtenido	Redirección 302; una visita con fecha de mañana y 'Control general APF'
Estado	Aprobado (R3, MySQL). Primera ejecución: Fallido
Evidencia	evidencias/03_correccion_MySQL/surefire-reports; incidencia INC-T01

9.1. Ampliación de casos

Conjunto	Técnica	Casos	Propósito
F01–F31	Funcional, HTTP real	31	Datos válidos, inválidos y límites de los seis flujos, con verificación de persistencia
N01–N08	Navegador automatizado	8 capturas	Recorrido visual de los seis flujos con dos rechazos
S01–S13	Seguridad básica, HTTP real	13	Autenticación existente, Actuator, XSS, inyección SQL, manipulación de identificadores, CSRF, cabeceras
E01	Exploratoria concurrente	1	Doble envío del alta de mascota (RN02, RN04)

10. Pruebas unitarias e integración

10.1. Ejecuciones realizadas

Las ejecuciones se registran por separado y no se suman entre sí. R0 corresponde al paquete recibido; R1 a R3 se ejecutaron en el ambiente consolidado.

Ejecución	Ejec.	Aprob.	Fall.	Casos fallidos	Duración Maven
Transferencia (Windows, H2)	6	4	2	CP04, CP06	26,590 s
R1 original Linux H2	6	4	2	CP04, CP06	01:46 min
R1b original MySQL	6	4	2	CP04, CP06	23,014 s
R2 corregido H2	6	6	0	-	27,330 s
R3 corregido MySQL	6	6	0	-	25,369 s

Fuente: logs y surefire-reports de evidencias/00_transferencia, 01_reproduccion_original, 01b_original_en_MySQL, 02_correccion_H2 y 03_correccion_MySQL. La duración de R1 incluye la descarga inicial de dependencias. R2 y R3 incluyen además la prueba de diagnóstico DIAG01.

10.2. Análisis de las fallas de CP04 y CP06

Las dos fallas se reprodujeron sin cambios en Linux con H2 y con MySQL; por tanto, no dependían del sistema operativo ni del motor de base de datos. En CP06, la respuesta 404 indicaba la ruta «owners/11/pets/visits/new»: el identificador de la mascota llegaba vacío.

El helper pet() de la prueba creaba una mascota, la asociaba a un propietario ya persistido y llamaba a saveAndFlush. Para una entidad existente, Spring Data ejecuta merge; la cascada MERGE persiste una copia gestionada de la mascota y la instancia creada por la prueba conserva el identificador nulo (Jakarta EE, 2022). Con ese identificador nulo, CP04 buscaba getPet(null) y CP06 construía una URL sin petId.

Para confirmar la hipótesis se añadió DIAG01, que replica el helper original y verifica que la instancia de la prueba queda con id nulo y fuera del contexto de persistencia, mientras la base de datos sí contiene la mascota con id asignado. DIAG01 aprobó en H2 y en MySQL. La causa se clasifica como problema del instrumento de prueba (INC-T01), no como defecto de Petclinic.

10.3. Corrección y comprobación

Se modificó únicamente el helper pet(): tras el flush, limpia el contexto, recarga el propietario y obtiene la mascota persistida, con dos aserciones de preparación. No se modificaron las aserciones de CP04 ni de CP06 ni el código de producción. El Anexo E reproduce el cambio.

Tras la corrección, CP01 a CP06 aprobaron en H2 (R2) y en MySQL 8.0.46 (R3). Los reportes originales se conservan en evidencias/00_transferencia y los nuevos en carpetas separadas.

10.4. Ejecución sobre MySQL

La configuración de pruebas del proyecto asocia el perfil mysql a un contenedor de Testcontainers, que requiere Docker. Al no disponer de Docker, se pasaron por línea de comandos las mismas propiedades de application-mysql.properties apuntando al MySQL local (AMB-02). El registro de R3 confirma el controlador MySQL Connector/J y el dialecto MySQLDialect.

10.5. Suite original del proyecto

Como referencia de estabilidad se ejecutó la suite completa en H2: 86 pruebas ejecutadas, 0 fallidas y 2 omitidas (MySqlIntegrationTests, por requerir Docker). Incluye las siete pruebas académicas; las demás pertenecen al proyecto y no se cuentan como trabajo del equipo.

10.6. Cobertura pendiente en este nivel

Los casos unitarios y de integración no comprueban todas las combinaciones de datos. La ampliación de validaciones, duplicados, fechas y edición se realizó en el nivel funcional (sección 11).

11. Pruebas funcionales

11.1. Enfoque y técnicas

Técnica	Descripción	Casos
Integración HTTP real	Script Python que envía solicitudes HTTP a la aplicación en ejecución (puerto 8080, MySQL), sigue redirecciones y consulta la página de detalle para verificar la persistencia.	F01–F31
Navegador automatizado	Chromium con Playwright: completa formularios y guarda capturas. Reproduce acciones de usuario, pero no equivale a una prueba manual.	N01–N08
MockMvc	Simula solicitudes dentro del contexto de Spring sin servidor HTTP.	CP06
Manual de navegador	No realizada en esta ejecución.	Pendiente del equipo

11.2. Precondiciones

Aplicación iniciada con el perfil mysql sobre MySQL 8.0.46 con los datos semilla del proyecto. Cada caso crea sus propios propietarios y mascotas con un sufijo único, de modo que ningún caso depende de otro. Fechas relativas al día de ejecución (29/10/2026): hoy, mañana y ayer.

11.3. Resultados

La tabla resume cada caso. Los pasos completos y la ruta de cada respuesta HTTP guardada se encuentran en la matriz y en evidencias/05b_funcionales_MySQL_corrida2/respuestas.

ID	Req.	Caso y datos	Esperado	Obtenido	Estado
F01	RF01	Registro de propietario con datos válidos. Datos: Ana Torres, Av. Central 123, Lima, 9876543210	302 a /owners/{id}; el detalle muestra los datos registrados	HTTP 302 -> http://localhost:8080/owners/46; detalle con datos=sí	Aprobado
F02	RF01	Registro rechazado: teléfono de 9 dígitos. Datos: teléfono='987654321'	200 con error en 'telephone'; el propietario no se guarda	HTTP 200; error telephone=sí; no guardado=sí	Aprobado
F03	RF01	Registro rechazado: teléfono de 11 dígitos. Datos: teléfono='98765432101'	200 con error en 'telephone'; el propietario no se guarda	HTTP 200; error telephone=sí; no guardado=sí	Aprobado
F04	RF01	Registro rechazado: teléfono con letras. Datos: teléfono='98765abcde'	200 con error en 'telephone'; el propietario no se guarda	HTTP 200; error telephone=sí; no guardado=sí	Aprobado
F05	RF01	Registro rechazado: dirección vacía y ciudad en blanco. Datos: address='', city=' '	200 con error en address y city	HTTP 200; address=sí; city=sí	Aprobado
F06	RF01	Límite de longitud del apellido (30/31). Datos: apellido de 30 y de 31 caracteres	30 caracteres: 302; 31 caracteres: 200 con error en lastName	30 -> HTTP 302; 31 -> HTTP 200, error=sí	Aprobado
F07	RF02	Búsqueda con coincidencia única. Datos: apellido=FgUCHIY	302 a /owners/48	HTTP 302 -> http://localhost:8080/owners/48	Aprobado
F08	RF02	Búsqueda por prefijo con varias coincidencias. Datos: prefijo=FhUCHIY (2 propietarios)	200 con lista que incluye ambos propietarios	HTTP 200; ambos listados=sí	Aprobado
F09	RF02	Búsqueda sin resultados. Datos: apellido=InexistenteUCHIY	200 con mensaje 'has not been found'	HTTP 200; mensaje=sí	Aprobado
F10	RF02	Búsqueda con espacios al inicio y al final. Datos: apellido=' FjUCHIY '	Se recortan espacios: 302 a /owners/51	HTTP 302 -> http://localhost:8080/owners/51	Aprobado
F11	RF02	Búsqueda con apellido vacío (todos). Datos: lastName=''	200 con listado paginado de propietarios	HTTP 200; listado paginado=sí	Aprobado
F12	RF03	Actualización válida de teléfono y ciudad. Datos: teléfono 9123456780, ciudad Arequipa	302 al detalle; se muestran los nuevos valores con el mismo id	HTTP 302 -> http://localhost:8080/owners/52; persistido=sí	Aprobado

ID	Req.	Caso y datos	Esperado	Obtenido	Estado
F13	RF03	Actualización rechazada: teléfono de 5 dígitos. Datos: teléfono=12345	200 con error en telephone; se conserva el teléfono anterior	HTTP 200; error=sí; sin cambios=sí	Aprobado
F14	RF03	Actualización rechazada: nombre vacío. Datos: firstName=''	200 con error en firstName; datos sin cambios	HTTP 200; error=sí; nombre conservado=sí	Aprobado
F15	RF04	Registro de mascota válida. Datos: Luna, 2022-05-10, cat	302; la mascota aparece asociada al propietario con sus datos	HTTP 302; petId=34	Aprobado
F16	RF04	Registro rechazado: nombre vacío. Datos: name=''	200 con error en name	HTTP 200; error name=sí	Aprobado
F17	RF04	Registro rechazado: nombre duplicado (distinta capitalización). Datos: Luna existente; nueva 'LUNA'	200 con error en name; sigue existiendo una sola mascota	HTTP 200; error=sí; mascotas con ese nombre=1	Aprobado
F18	RF04	Registro rechazado: nacimiento futuro (mañana). Datos: birthDate=2026-10-30	200 con error en birthDate	HTTP 200; error birthDate=sí	Aprobado
F19	RF04	Límite: fecha de nacimiento igual a hoy. Datos: birthDate=2026-10-29	302 (la regla solo rechaza fechas posteriores a hoy)	HTTP 302	Aprobado
F20	RF04	Límite de longitud del nombre de mascota (30/31). Datos: nombre de 30 y de 31 caracteres	30: 302; 31: 200 con error en name	30 -> HTTP 302; 31 -> HTTP 200, error=sí	Aprobado
F21	RF04	Registro rechazado: sin fecha de nacimiento. Datos: birthDate=''	200 con error en birthDate	HTTP 200; error birthDate=sí	Aprobado
F22	RF05	Edición válida de nombre, tipo y fecha. Datos: Luna->Lunita, cat->dog, 2021-03-15	302; la misma mascota (mismo id) muestra los nuevos datos	HTTP 302; mismo id=sí	Aprobado
F23	RF05	Edición rechazada: renombrar a un nombre ya usado. Datos: Max -> Luna (existente)	200 con error en name; Max conserva su nombre	HTTP 200; error=sí; Max sin cambios=sí	Aprobado
F24	RF05	Edición rechazada: fecha de nacimiento futura. Datos: birthDate=2026-10-30	200 con error en birthDate; se conserva la fecha anterior	HTTP 200; error=sí; fecha conservada=sí	Aprobado
F25	RF05	Edición rechazada: nombre vacío (persistencia, complementa CP05). Datos: name=''	200 con error en name; la mascota conserva su nombre	HTTP 200; error=sí; conservado=sí	Aprobado
F26	RF06	Registro de visita válida (mañana). Datos: fecha=2026-10-30, 'Control general'	302 al propietario; la visita aparece en la mascota	HTTP 302; visita visible=sí	Aprobado
F27	RF06	Visita rechazada: fecha de hoy (límite). Datos: fecha=2026-10-29	200 con error en date; no se registra la visita	HTTP 200; error date=sí; no registrada=sí	Aprobado
F28	RF06	Visita rechazada: fecha pasada. Datos: fecha=2026-10-28	200 con error en date; no se registra la visita	HTTP 200; error date=sí; no registrada=sí	Aprobado
F29	RF06	Visita rechazada: descripción vacía. Datos: descripción=''	200 con error en description; no se registra la visita	HTTP 200; error description=sí; no registrada=sí	Aprobado
F30	RF06	Visita rechazada: descripción solo con espacios. Datos: descripción=' '	200 con error en description; no se registra la visita	HTTP 200; error description=sí; no registrada=sí	Aprobado
F31	RF06	Límite de longitud de la descripción (255/256). Datos: descripción de 255 y de 256 caracteres	255: 302; 256: 200 con error en description	255 -> HTTP 302; 256 -> HTTP 200, error=sí	Aprobado

Fuente: evidencias/05b_funcionales_MySQL_corrida2/resultados.json (corrida 2). La corrida 1 (05a) registró 29 aprobados y 2 fallidos por un problema del script (INC-T02).

11.4. Recorrido en navegador automatizado

Las capturas N01 a N08 recorren los seis flujos: registro rechazado por teléfono de nueve dígitos y registro válido, búsqueda, actualización del teléfono, alta de mascota, rechazo de fecha de nacimiento futura, edición de mascota y registro de visita.

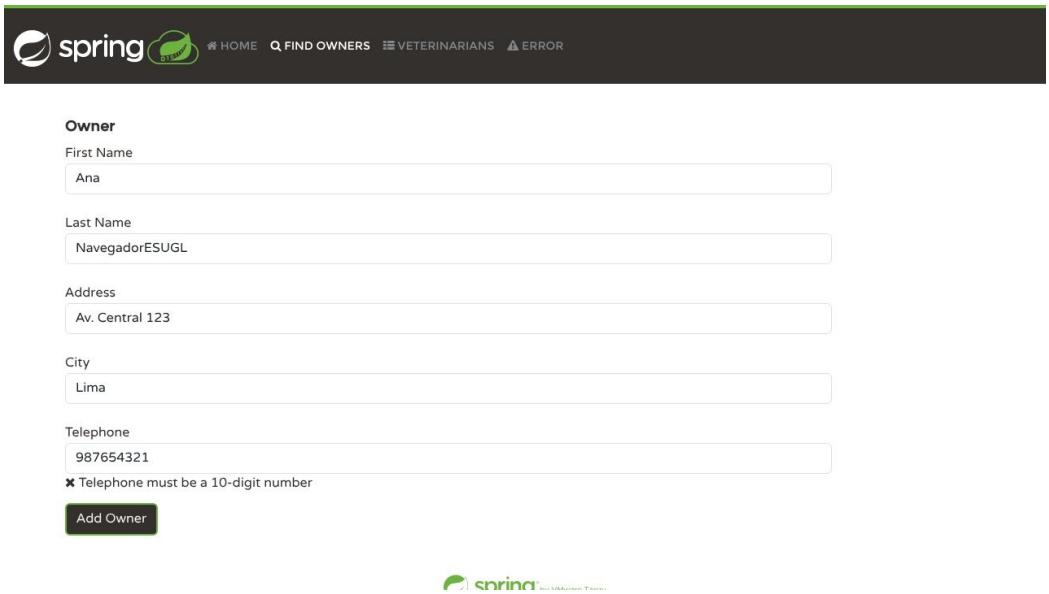


Figura 1. N01: el formulario devuelve el error «Telephone must be a 10-digit number» (RF01).

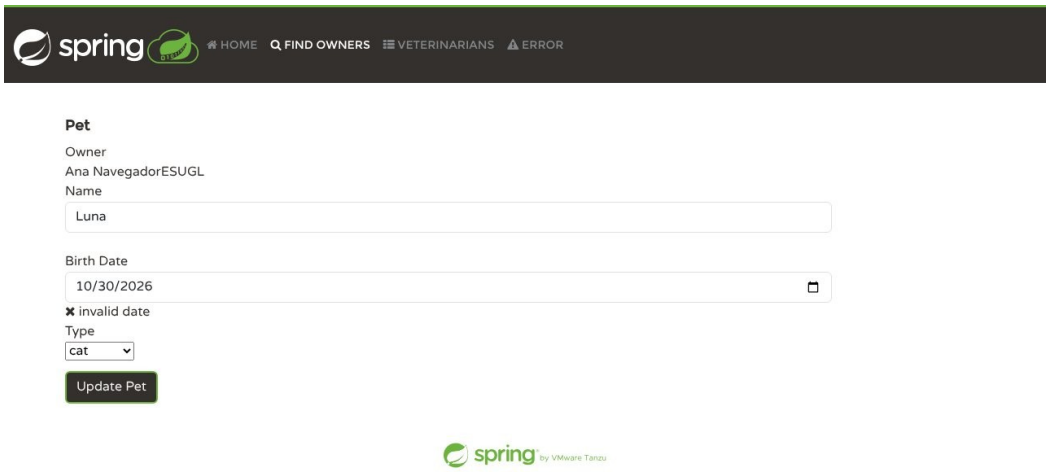


Figura 2. N06: la edición con fecha de nacimiento de mañana se rechaza con «invalid date» (RF05).

11.5. Prueba exploratoria de concurrencia (E01)

Se envió en paralelo el alta de la mascota «Gemela» cuatro veces por propietario, situación equivalente a un doble envío del formulario. El resultado esperado era una mascota por propietario, el rechazo controlado de las demás solicitudes y ninguna mascota sin propietario.

Ejecución	Propietarios	Respuestas 302	Rechazo controlado	HTTP 500	Mascotas sin propietario
MySQL, corrida 1	10	12	0	28	2
MySQL, corrida 2	20	27	2	51	7
H2 (modo archivo)	20	38	42	0	18

Fuente: resultado.json de evidencias/07, 07b y 07c; conteo de filas con owner_id nulo consultado directamente en cada base.

Cada respuesta 302 adicional a la primera dejó una mascota desvinculada. La traza SQL de MySQL (evidencias/07d) registra que la segunda solicitud ejecuta «update pets set owner_id=null where owner_id=… and id=…»: al fusionar su copia del propietario, desvincula

la mascota que otra solicitud acababa de confirmar. El usuario recibe un mensaje de éxito para un registro que deja de ser visible, lo que incumple RN02 (DEF-P02).

En MySQL, además, la violación de unicidad se informa con la clave «pets.owner_id», mientras el controlador reconoce solo «unique_owner_pet_name», nombre definido en el esquema H2. Por ello la solicitud duplicada produce HTTP 500 en lugar del mensaje «already exists» (DEF-P03). E01 se registra como fallido.

11.6. Análisis

En flujo secuencial, los 31 casos funcionales coinciden con las reglas RN01 a RN07: se rechazan teléfonos de 9 y 11 dígitos o con letras, campos vacíos, nombres duplicados aun con distinta capitalización, fechas futuras de nacimiento y visitas de hoy o pasadas. Los límites de 30 caracteres (apellido y nombre de mascota) y 255 (descripción) se aceptan, y uno más se rechaza. La fecha de nacimiento igual a hoy se acepta, coherente con RN05.

Los defectos aparecen fuera del flujo secuencial: en envíos simultáneos (E01) y en el manejo de identificadores no asociados (sección 13).

12. Pruebas de rendimiento básico

12.1. Escenario

Se modeló el trabajo de recepción de la clínica: búsqueda de propietarios por apellido (peso 4), consulta del detalle de un propietario semilla (peso 4), apertura del formulario de búsqueda (peso 1) y registro de una visita en una mascota propia de cada usuario virtual (peso 1). Cada usuario crea su propietario y mascota al iniciar y espera entre 1 y 2 segundos entre acciones.

12.2. Configuración

Parámetro	Valor
Herramienta	Locust 2.47.0, modo sin interfaz (scripts/locustfile.py)
Aplicación	JAR del commit evaluado, perfil mysql, JVM -Xmx512m, reinicio previo a la medición
Base de datos	MySQL 8.0.46 local
Calentamiento	60 s con 10 usuarios; resultados guardados y excluidos de las métricas
Carga nominal	10 usuarios, incremento de 2 usuarios/s, 180 s (19:19:47 a 19:22:47)
Carga elevada	30 usuarios, incremento de 3 usuarios/s, 180 s (19:22:56 a 19:25:57)
Equipo	Aplicación, base de datos y generador en el mismo contenedor de 2 vCPU
Umbrales	Propuestos por el equipo, no exigidos por el docente: p95 < 1000 ms y errores < 1 %

12.3. Resultados

Escenario	Solicitudes	Errores	Mediana	Promedio	p95	p99	Máximo	Solic./s
10 usuarios	1212	0	11 ms	12 ms	21 ms	29 ms	87 ms	6,74
30 usuarios	3592	0	8 ms	8 ms	16 ms	26 ms	86 ms	19,96

Fuente: resumen final de consola.log de Locust en evidencias/08_rendimiento_MySQL. Detalle por operación en la hoja «Rendimiento» de la matriz y en reporte.html.

Operación	Solic. (10 u.)	p95 (10 u.)	Solic. (30 u.)	p95 (30 u.)	Máx. (30 u.)
GET /owners?lastName	464	14 ms	1378	11 ms	38 ms
GET /owners/[id]	457	20 ms	1418	15 ms	86 ms
GET /owners/find	127	11 ms	356	9 ms	22 ms
POST /visits/new	134	28 ms	350	19 ms	30 ms

12.4. Interpretación

No se registraron errores en 4804 solicitudes medidas. El rendimiento creció de 6,74 a 19,96 solicitudes por segundo, en proporción al número de usuarios, y el p95 se mantuvo entre 16 y 21 ms. Ambos escenarios cumplen los umbrales propuestos por el equipo.

El p95 fue menor con 30 usuarios que con 10. La causa más probable es el calentamiento progresivo de la JVM y de la caché de la aplicación durante la primera medición; no debe interpretarse como mejora por mayor carga. La memoria residente del proceso Java se mantuvo entre 386 y 390 MiB durante la carga elevada.

La carga es pequeña y local. La prueba no determina la capacidad máxima, el comportamiento con red real ni con datos voluminosos, y no sustituye una prueba en un entorno equivalente a producción.

13. Pruebas de seguridad básica

13.1. Alcance

Las pruebas se ejecutaron solo contra la instancia local (el script rechaza otros hosts). No incluyeron fuerza bruta, denegación de servicio ni herramientas de explotación. Se examinaron la existencia de autenticación y autorización, la validación de entradas y límites, la asociación propietario-mascota, la exposición de información y controles de configuración.

13.2. Autenticación y autorización

El pom.xml no incluye Spring Security y no existe una ruta de inicio de sesión (/login responde 404). Todas las rutas de consulta y modificación responden 200 a solicitudes anónimas. Por tanto, no hay cuentas, roles ni controles de autorización que probar. No se añadió un módulo de autenticación, porque no forma parte de RF01–RF06. S01 se registra como observación de alcance.

13.3. Resultados

ID	Verificación	Esperado	Obtenido	Estado	Incid.
S01	Verificación de autenticación existente	Determinar si la variante exige autenticación (no se asume un resultado)	/=200; /owners/find=200; /owners?lastName==200; /owners/1=200; /owners/1/edit=200; /vets.html=200; /owners/new=200; /login=404	Observación	OBS-01
S02	Exposición de endpoints de administración (Actuator)	Endpoints sensibles no accesibles de forma anonima	/actuator=200; /actuator/env=200; /actuator/configprops=200; /actuator/beans=200; /actuator/mappings=200; /actuator/heapdump(HEAD)=404	Fallido	SEC-01
S03	XSS almacenado en dirección y ciudad	El contenido se muestra escapado; no aparece una etiqueta <script> ejecutable	etiqueta sin escapar=no; escapada=sí	Aprobado	-
S04	Inyección SQL en lastName	Se tratan como texto: 200 'has not been found', sin trazas ni efectos	[' OR '1'='1] HTTP 200 not_found=sí; [x %' OR 1=1 --] HTTP 200 not_found=sí; ['; DROP TABLE owners; --] HTTP 200 not_found=sí; tabla owners operativa=sí	Aprobado	-
S05	Asignación masiva: id=1 enviado al crear propietario	Se crea un propietario nuevo; el propietario 1 no se modifica	HTTP 302; nuevo id=73; propietario 1 sin cambios=sí	Aprobado	-
S06	Manipulación del id en la actualización del propietario	El propietario B no se modifica	HTTP 302 -> http://localhost:8080/owners/74; B modificado=no; A modificado=sí	Aprobado	-
S07	Visita a una mascota que no pertenece al propietario de la URL	Se rechaza la operación; no se registra la visita; sin traza interna	GET HTTP 500; POST HTTP 500; visita registrada=no; traza=no	Aprobado	OBS-02
S08	Edición de una mascota que pertenece a otro propietario	Se rechaza con un error controlado; la mascota de B no cambia; no se crean registros	GET HTTP 500; POST HTTP 500; mascota B modificada=no; A recibio mascota=no; traza=no	Aprobado	DEF-P01
S09	Propietario inexistente y divulgación en la página de error	Error controlado sin traza de pila	HTTP 500; traza=no; mensaje interno visible=no	Aprobado	OBS-02
S10	Entradas sobredimensionadas	200 con error de validación; sin error 500 ni almacenamiento	256 -> 200/True; ciudad 81 -> 200/True; 20000 -> 200/True	Aprobado	-
S11	Protección CSRF en formularios	Si la aplicación protegiera contra CSRF, el POST sin token se rechazaria	token en formulario=no; POST HTTP 302	Fallido	SEC-02
S12	Cabeceras de seguridad HTTP en la página principal	Presencia de cabeceras defensivas básicas	X-Content-Type-Options=no; X-Frame-Options=no; Content-Security-Policy=no; Cache-Control=no	Fallido	SEC-03
S13	Identificador de sesión expuesto en la URL de redirección	La redirección no incluye el identificador de sesión	HTTP 302; Location contiene jsessionid=sí	Fallido	SEC-04

Fuente: evidencias/06_seguridad_MySQL/resultados.json y respuestas HTTP guardadas.

13.4. Análisis

Controles que funcionan. Thymeleaf escapa el contenido almacenado (S03). La búsqueda trata las cargas de inyección SQL como texto (S04). La lista de campos no enlazables impide fijar el id al crear o actualizar un propietario (S05, S06). Las entradas sobredimensionadas se rechazan con error de validación (S10).

Asociación propietario-mascota. No fue posible registrar una visita (S07) ni modificar una mascota de otro propietario (S08). La integridad se conserva, pero la respuesta es HTTP 500. En S08 el error no está controlado: findPet() devuelve null y el controlador falla con NullPointerException (DEF-P01). En S07 y S09 la excepción se lanza a propósito, aunque el código HTTP adecuado sería 404 (OBS-02). Las páginas de error no exhiben trazas internas.

Configuración. Los endpoints de Actuator están expuestos sin autenticación (SEC-01); /actuator/env enmascara los valores y heapdump no está disponible, pero se publican beans, mapeos, configuración y volcado de hilos. La propia configuración indica que esa exposición es solo para desarrollo. Tampoco hay protección CSRF (SEC-02), faltan cabeceras defensivas (SEC-03) y la primera redirección incluye el identificador de sesión (SEC-04).

Estos resultados no permiten afirmar que el sistema sea seguro. Indican qué controles básicos existen en esta variante de demostración y cuáles deberían añadirse antes de un despliegue con usuarios reales.

14. Pruebas móviles, de usabilidad y accesibilidad

Corresponden al sprint 5 y al tercer avance. No se presentan resultados en este documento.

15. Pruebas de regresión

Corresponden al sprint 5. En este avance solo se reejecutaron los casos corregidos para comprobar las correcciones (secciones 10.3 y 16).

16. Registro de incidencias y correcciones

Las incidencias se separan en cinco categorías: defectos del producto (DEF), hallazgos de seguridad (SEC), observaciones de alcance y calidad (OBS), problemas del instrumento de prueba (INC-T) y problemas del ambiente (AMB). Los defectos del producto quedan abiertos porque el alcance excluye modificar el código de producción; se incluye una corrección propuesta.

ID	Categoría	Casos	Req.	Severidad	Prioridad	Estado
INC-T01	Problema del instrumento de prueba	CP04, CP06	RF04, RF06	Alta (invalida dos casos)	Alta	Cerrado
INC-T02	Problema del instrumento de prueba	F01, F11	RF01, RF02	Media	Alta	Cerrado
DEF-P01	Defecto del producto (robustez)	S08	RF05	Media	Media	Abierto
DEF-P02	Defecto del producto (integridad bajo concurrencia)	E01	RF04 (RN02, RN04)	Alta	Media	Abierto
DEF-P03	Defecto del producto (manejo de errores, solo MySQL)	E01	RF04 (RN04)	Media	Media	Abierto
SEC-01	Hallazgo de seguridad (configuración)	S02	Transversal	Media (local); alta si se despliega así	Media	Abierto
SEC-02	Hallazgo de seguridad (control ausente)	S11	Transversal	Media	Baja	Abierto
SEC-03	Hallazgo de seguridad (configuración)	S12	Transversal	Baja	Baja	Abierto
SEC-04	Hallazgo de seguridad (configuración)	S13	Transversal	Baja	Baja	Abierto
OBS-01	Observación de alcance	S01	Transversal	No aplica	No aplica	Registrado
OBS-02	Hallazgo de calidad (manejo de errores)	S07, S09	RF02, RF06	Baja	Baja	Abierto
AMB-01	Problema de ambiente	Todos	-	Alta (bloqueante)	Alta	Cerrado
AMB-02	Problema de ambiente	CP02–CP04, CP06	-	Media	Alta	Cerrado (mitigado)
AMB-03	Problema de ambiente	Todos (Windows)	-	Media	Alta	Cerrado

Detalle completo de reproducción, esperado, obtenido, corrección y comprobación en la hoja «Incidencias» de la matriz.

En el alcance funcional secuencial (F01–F31) no se identificaron defectos del producto; sus dos fallas iniciales correspondieron al script (INC-T02).

INC-T01 · Problema del instrumento de prueba

Campo	Descripción
Caso / requisito	CP04, CP06 / RF04, RF06
Reproducción	Ejecutar AvanceIntegrationTests original. El helper pet() devuelve la instancia Pet creada por el test. Como el propietario ya existe, saveAndFlush ejecuta merge y la cascada persiste una copia gestionada; la instancia del test conserva id nulo.
Esperado	El fixture entrega una mascota con el id asignado por la base de datos
Obtenido	CP04: getPet(null) devuelve nulo. CP06: URL /owners/{id}/pets//visits/new y HTTP 404 ('No static resource owners/11/pets/visits/new')
Severidad / prioridad	Alta (invalida dos casos) / Alta

Campo	Descripción
Estado	Cerrado
Corrección	pet() recarga la mascota desde la base tras flush y clear y verifica que tenga id; las aserciones de CP04 y CP06 no se modificaron
Comprobación posterior	R2 (H2) y R3 (MySQL): CP01–CP06 aprobados. DIAG01 confirma que la instancia original queda con id nulo
Evidencia	01_reproduccion_original, 01b_original_en_MySQL, 02_correccion_H2, 03_correccion_MySQL

INC-T02 · Problema del instrumento de prueba

Campo	Descripción
Caso / requisito	F01, F11 / RF01, RF02
Reproducción	Primera corrida de funcionales_http.py. F01: la primera redirección incluye ';jsessionid=...' y la expresión regular no la admitía. F11: el script buscaba un marcador inexistente en el listado.
Esperado	El script reconoce la redirección y el listado paginado que la aplicación sí devuelve
Obtenido	F01 y F11 marcados como fallidos aunque la aplicación respondió correctamente (302 a /owners/20; tabla id='owners' con páginas)
Severidad / prioridad	Media / Alta
Estado	Cerrado
Corrección	Se admite ';jsessionid' al extraer el id y se verifica la tabla id='owners' y el enlace a la página 2
Comprobación posterior	Corrida 2: 31 de 31 casos aprobados
Evidencia	05a_funcionales_MySQL_corrida1, 05b_funcionales_MySQL_corrida2

DEF-P01 · Defecto del producto (robustez)

Campo	Descripción
Caso / requisito	S08 / RF05
Reproducción	GET o POST /owners/{A}/pets/{idMascotaDeB}/edit. findPet() devuelve null cuando la mascota no pertenece al propietario de la URL.
Esperado	Rechazo controlado (por ejemplo, 404), como hace VisitController al validar la pertenencia
Obtenido	HTTP 500 con cuerpo vacío. GET: error de plantilla (EL1012E, pet nulo). POST: NullPointerException en PetController.java:148. La mascota de B no se modifica
Severidad / prioridad	Media / Media
Estado	Abierto
Corrección	Propuesta, no aplicada (no se modifica código de producción): validar el resultado de findPet() y lanzar una excepción asociada a 404
Comprobación posterior	Pendiente de corrección por el responsable del producto
Evidencia	06_seguridad_MySQL/respuestas/S08_*.http.txt; registro del servidor

DEF-P02 · Defecto del producto (integridad bajo concurrencia)

Campo	Descripción
Caso / requisito	E01 / RF04 (RN02, RN04)

Campo	Descripción
Reproducción	Enviar en paralelo 4 altas de la misma mascota para un propietario nuevo (equivale a un doble envío del formulario).
Esperado	Una mascota; las demás solicitudes rechazadas; ninguna mascota sin propietario
Obtenido	Más de una respuesta 302 por propietario. La segunda solicitud ejecuta 'update pets set owner_id=null ...' y desvincula la mascota confirmada. MySQL: 2 huérfanas (10 prop.) y 7 (20 prop.). H2: 18 huérfanas (20 prop.)
Severidad / prioridad	Alta / Media
Estado	Abierto
Corrección	Propuesta: control de concurrencia optimista (@Version) en Owner o asociación gestionada desde Pet; como mitigación, impedir el doble envío en la interfaz
Comprobación posterior	Pendiente; repetir E01 tras la corrección
Evidencia	07_exploratorio_duplicado_MySQL, 07b_..._repeticion, 07c_..._H2, 07d_traza_sql_MySQL

DEF-P03 · Defecto del producto (manejo de errores, solo MySQL)

Campo	Descripción
Caso / requisito	E01 / RF04 (RN04)
Reproducción	Mismo escenario que DEF-P02 sobre MySQL. La violación de unicidad se reporta con la clave 'pets.owner_id'.
Esperado	Mensaje 'already exists' en el campo name, como ocurre con H2
Obtenido	HTTP 500 en 28 de 40 y 51 de 80 solicitudes. isDuplicatePetNameViolation() busca 'unique_owner_pet_name', nombre que solo existe en el esquema H2
Severidad / prioridad	Media / Media
Estado	Abierto
Corrección	Propuesta: nombrar la restricción en db/mysql/schema.sql o detectar la violación por código de error
Comprobación posterior	Pendiente
Evidencia	07_exploratorio_duplicado_MySQL, 07b_..._repeticion; registro del servidor

SEC-01 · Hallazgo de seguridad (configuración)

Campo	Descripción
Caso / requisito	S02 / Transversal
Reproducción	GET anónimo a /actuator, /actuator/env, /configprops, /beans, /mappings
Esperado	Endpoints de administración no accesibles sin autenticación
Obtenido	HTTP 200 en todos. /actuator/env enmascara valores (******); heapdump no disponible (404). Se exponen beans, mapeos, configuración, threaddump y sbom
Severidad / prioridad	Media (local); alta si se despliega así / Media
Estado	Abierto
Corrección	Propuesta: limitar management.endpoints.web.exposure.include a health,info fuera de desarrollo
Comprobación posterior	Pendiente
Evidencia	06_seguridad_MySQL/respuestas/S02_*

17. Métricas y análisis de resultados

17.1. Resultados de la última versión

Conjunto	Ejecutados	Aprobados	Fallidos	Bloqueados	Observación
Unitarias e integración	6	6	0	0	0
Funcionales HTTP	31	31	0	0	0
Seguridad básica	13	8	4	0	1
Exploratoria de concurrencia	1	0	1	0	0
Total	**51**	**45**	**5**	**0**	**1**

Sobre 50 casos evaluables (se excluye la observación S01), la tasa de aprobación es 90,0 %. Los 5 fallidos corresponden a la prueba exploratoria E01 y a cuatro controles de seguridad ausentes (S02, S11, S12, S13). Ninguno de los 37 casos planificados para RF01–RF06 (CP y F) falla en la última versión.

17.2. Ejecuciones históricas

Conjunto	Ejecución	Ejecutados	Aprobados	Fallidos	Motivo de las fallas
CP01–CP06	R0 transferencia y R1, R1b originales	6 por ejecución	4	2	INC-T01 (fixture)
CP01–CP06	R2 (H2) y R3 (MySQL) corregidos	6 por ejecución	6	0	-
F01–F31	Corrida 1	31	29	2	INC-T02 (script)
F01–F31	Corrida 2	31	31	0	-

17.3. Incidencias por categoría

Categoría	Total	Abiertas	Cerradas o registradas
Defectos del producto (DEF)	3	3	0
Hallazgos de seguridad (SEC)	4	4	0
Observaciones (OBS)	2	1	1 registrada
Instrumento de prueba (INC-T)	2	0	2
Ambiente (AMB)	3	0	3

Severidad de los defectos del producto: uno alta (DEF-P02) y dos media (DEF-P01, DEF-P03).

17.4. Tiempos

Medición	Valor	Fuente
Ejecución Maven de CP01–CP06 corregidos (H2 / MySQL)	27,330 s / 25,369 s	02 y 03, ejecucion.log
Suite completa del proyecto (H2)	36,896 s	04, ejecucion.log
Funcionales F01–F31, corrida 2	2 s	05b, resultados.json
Seguridad S01–S13	1 s	06, resultados.json
Rendimiento: p95 con 10 y 30 usuarios	21 ms y 16 ms	08, consola.log

17.5. Interpretación

Las validaciones de RF01 a RF06 se comportan según las reglas definidas cuando las operaciones se ejecutan una a una, tanto en H2 como en MySQL. Las fallas iniciales de CP04, CP06, F01 y F11 se debieron a los instrumentos de prueba; su análisis evitó registrar como defectos del producto problemas que no lo eran.

Los defectos confirmados se concentran en situaciones no cubiertas por los casos iniciales: solicitudes simultáneas sobre el mismo propietario y acceso a mascotas no asociadas. DEF-P02 es el más relevante, porque el sistema confirma una operación cuyo resultado no conserva. Los hallazgos de seguridad describen controles ausentes en una aplicación de demostración y deben valorarse según el entorno de despliegue.

Las métricas se basan en 51 casos y dos escenarios de carga locales. No permiten generalizar sobre funcionalidades no seleccionadas ni sobre el comportamiento en producción.

18. Conclusiones

Las conclusiones finales corresponden al informe de la semana 18. La interpretación parcial de este avance se presenta en la sección 17.5.

19. Referencias

Jakarta EE. (2022). Jakarta Persistence 3.1 specification. Eclipse Foundation. https://jakarta.ee/specifications/persistence/3.1/

Locust. (2026). Locust documentation. https://docs.locust.io/

Microsoft. (2026). Playwright for Python. https://playwright.dev/python/

Minhas, N. M., Petersen, K., Börstler, J., & Wnuk, K. (2020). Regression testing for large-scale embedded software development—Exploring the state of practice. Information and Software Technology, 120, 106254. https://doi.org/10.1016/j.infsof.2019.106254

Noël, R., Taramasco, C., & Márquez, G. (2022). Standards, processes, and tools used to evaluate the quality of health information systems: Systematic literature review. Journal of Medical Internet Research, 24(3), e26577. https://doi.org/10.2196/26577

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Shin, Y., Choi, Y., & Lee, W. J. (2013). Integration testing through reusing representative unit test cases for high-confidence medical software. Computers in Biology and Medicine, 43(5), 434–443. https://doi.org/10.1016/j.compbiomed.2013.01.024

Spring. (2026). Spring Boot reference documentation: Endpoints. https://docs.spring.io/spring-boot/reference/actuator/endpoints.html

Spring Projects. (2026). spring-petclinic [Software]. GitHub. https://github.com/spring-projects/spring-petclinic

Tran, H. K. V., Unterkalmsteiner, M., Börstler, J., & Ali, N. bin. (2021). Assessing test artifact quality—A tertiary study. Information and Software Technology, 139, 106620. https://doi.org/10.1016/j.infsof.2021.106620

20. Anexos

Anexo A. Organización de evidencias

Las carpetas se encuentran en el repositorio del equipo (https://github.com/Mooneer17/spring-petclinic-apf2) y en la carpeta de entrega.

Carpeta	Contenido
00_transferencia	Logs y reportes originales recibidos (Windows, H2). No modificados.
01_reproduccion_original / 01b_original_en_MySQL	Pruebas originales sin cambios en H2 y MySQL (R1, R1b).
02_correccion_H2 / 03_correccion_MySQL	CP01–CP06 corregidos y DIAG01 (R2, R3).
04_suite_completa_H2	Suite completa del proyecto.
05a / 05b_funcionales_MySQL	Corridas 1 y 2 de F01–F31: resultados.json y respuestas HTTP.
06_seguridad_MySQL	S01–S13: resultados.json y respuestas HTTP.
07, 07b, 07c, 07d	E01 en MySQL y H2; consultas de filas huérfanas; traza SQL.
08_rendimiento_MySQL	Calentamiento y cargas de 10 y 30 usuarios: consola, CSV, reporte HTML y memoria.
09_navegador_MySQL	Capturas N01–N08 y recorrido.json.
AMBIENTE.txt	Versiones de sistema, Java, Maven, MySQL, herramientas y hash del JAR.

Anexo B. Pasos de reproducción

Desde la carpeta proyecto/, con un JDK 17 o superior y acceso a Maven Central:

```
# Unitarias e integración en H2
./mvnw test -Dtest='AvanceUnitTests,AvanceIntegrationTests,DiagnosticoFixtureTests' -
Dsurefire.failIfNoSpecifiedTests=false
# Las mismas pruebas sobre MySQL local (usuario y base de db/mysql/user.sql)
./mvnw test -Dtest='Avance*,DiagnosticoFixtureTests' -
Dsurefire.failIfNoSpecifiedTests=false \
  -Ddatabase=mysql -Dspring.datasource.url=jdbc:mysql://localhost:3306/petclinic \
  -Dspring.datasource.username=petclinic -Dspring.datasource.password=petclinic -
Dspring.sql.init.mode=always
# Aplicación sobre MySQL y pruebas HTTP (desde scripts/)
./mvnw package -DskipTests && java -jar target/spring-petclinic-4.0.0-SNAPSHOT.jar --
spring.profiles.active=mysql
python3 funcionales_http.py --out salida/funcionales --bd MySQL
python3 seguridad_http.py --out salida/seguridad --bd MySQL
python3 exploratorio_duplicado_concurrente.py --out salida/e01 --bd MySQL --owners 20
locust -f locustfile.py --headless -H http://localhost:8080 -u 10 -r 2 -t 180s --csv
salida/carga10
python3 navegador_flujos.py --out salida/navegador
```

En Linux, scripts/ejecutar_apf2.sh automatiza estos pasos; en Windows, herramientas/ejecutar_apf2.ps1. La guía GUIA_REPRODUCCION.md detalla requisitos y salidas esperadas.

Anexo C. Versiones

Sistema evaluado: repositorio oficial spring-projects/spring-petclinic, commit 500158f732419217507c7656904b8e6aa1bcc0d6; Spring Boot 4.1.0; java.version 17. Repositorio del equipo con las pruebas, scripts y evidencias: https://github.com/Mooneer17/spring-petclinic-apf2. Detalle de herramientas en evidencias/AMBIENTE.txt.

Anexo D. Capturas del navegador automatizado

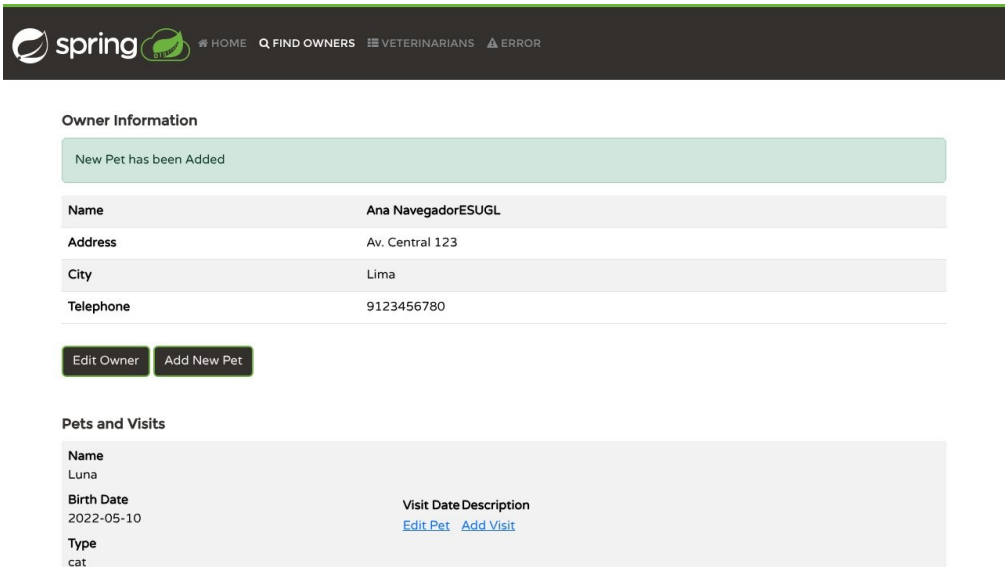


Figura 3. N05: mascota Luna registrada y asociada al propietario (RF04).

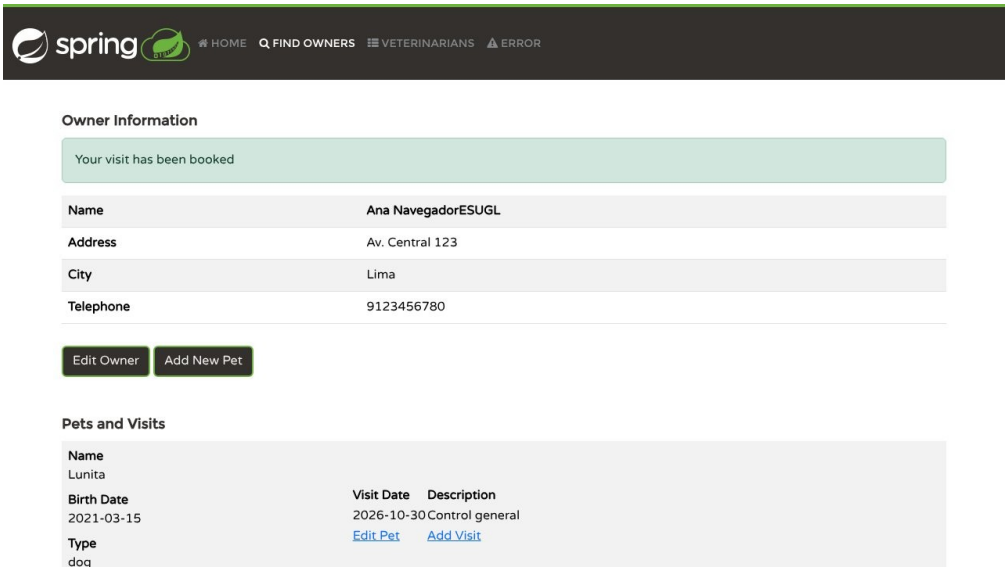


Figura 4. N08: visita del 30/10/2026 registrada para la mascota editada (RF06).

Anexo E. Corrección del helper pet() (INC-T01)

```
private Pet pet(Owner o) {
    Pet p = new Pet();
    p.setName("LunaAPF");
    p.setBirthDate(LocalDate.of(2022, 5, 10));
    p.setType(types.findPetTypes().iterator().next());
    o.addPet(p);
    owners.saveAndFlush(o);
    em.clear();                          // nuevo
    Pet persisted = owners.findById(o.getId()).orElseThrow()
            .getPet("LunaAPF", true);  // nuevo: mascota persistida
    assertThat(persisted).isNotNull();
    assertThat(persisted.getId()).isNotNull();
    return persisted;                    // antes: return p (id nulo)
}
```